EXPOFRITZ ACESSÍVEL: APLICATIVO PARA PESSOAS COM DEFICIÊNCIA VISUAL

Leonardo Oliani Fernandes, Dalton Solano dos Reis – Orientador

Curso de Bacharel em Ciência da Computação
Departamento de Sistemas e Computação
Universidade Regional de Blumenau (FURB) – Blumenau, SC – Brasil

lofernandes@furb.br, dalton@furb.br

Resumo: *Este trabalho visa o desenvolvimento de um aplicativo móvel acessível voltado à identificação de objetos físicos em ambientes de acervo, com foco na inclusão de pessoas com deficiência visual. A solução utiliza a câmera do dispositivo para capturar imagens em tempo real e aplica um modelo de rede neural convolucional MobileNetV2, treinado com TensorFlow, para realizar a classificação dos objetos. O modelo foi convertido para o formato compatível com a biblioteca Barracuda e integrado ao Unity, permitindo a inferência diretamente no dispositivo, sem necessidade de conexão com a internet. A aplicação fornece feedback imediato ao usuário por meio de síntese de voz, indicando o objeto detectado. Os testes realizados demonstraram precisão satisfatória no reconhecimento de diferentes objetos, confirmando a viabilidade do uso da inteligência artificial embarcada em dispositivos móveis como ferramenta de acessibilidade.*

Palavras-chave*: Acessibilidade. Aplicativo móvel. Visão computacional. Reconhecimento de objetos. Inteligência artificial.*

1 INTRODUÇÃO

Existem cerca de 36,5 milhões de pessoas com deficiência visual grave no Brasil (IBGE, 2010), ou seja, aproximadamente 18,6% da população brasileira da época. Esse grupo infelizmente, mesmo com os avanços tecnológicos, ainda sente muita dificuldade para se obter acesso à educação e cultura por conta da falta de acessibilidade, impactando em suas construções sociais e culturais. Um grande exemplo de espaço cultural no qual ainda se encontram muitos obstáculos é em museus e exposições culturais, principalmente se considerar a interação com as obras de maneira autônoma (Fairbanks, 2023).

Uma das principais e mais conhecidas tecnologias de integração para os cegos é o Sistema de Braille, que apesar da sua importância apresenta muitas limitações, pois muitos deficientes visuais não chegam nem perto da oportunidade de aprender o método de leitura com o Braille e os que conseguem aprender ainda assim continuam com o acesso limitado a imagens, quadros e a cultura no geral (Torres; Mazzoni; Mello, 2007). Essa realidade, demonstra que, apesar dos avanços, ainda vivemos em um contexto primitivo no que diz respeito à acessibilidade. É fundamental reconhecer que o deficiente visual não deve ser tratado como alguém "invalido", mas sim, como um cidadão como qualquer outro, que necessita de recursos acessíveis para exercer plenamente seus direitos, especialmente no acesso à cultura (Montenegro, 2015).

A visão computacional é um campo da ciência que permite a um computador "ver" o ambiente ao seu redor, extraindo informações relevantes de imagens capturadas por câmeras, sensores e outros dispositivos (Neves; Vieira Neto; Gonzaga, 2012). A partir desses dados, o sistema consegue reconhecer, analisar e interpretar os objetos presentes na imagem. Embora seja uma tecnologia relativamente nova, os primeiros sinais desta abordagem surgiram em 1955, quando Selfridge falou da necessidade de dar aos computadores "olhos e ouvidos". No entanto, foi somente na década de 1970 que começaram os primeiros experimentos combinando visão computacional e inteligência artificial (Milano; Honorato, 2010). Nesse contexto, a Áudio Descrição (AD) surge como recurso essencial da tecnologia assistiva, transformando imagens em significados auditivos em ambientes educacionais, culturais e sociais. Assim aumentando a acessibilidade da comunicação para todos (Oliveira; Moraes, 2023).

A Exposição Científica Fritz Müller, inaugurada na Universidade Regional de Blumenau (FURB) em 2023, é um espaço interativo e educativo que proporciona aos visitantes uma verdadeira viagem pelos biomas do planeta. Dividida em setores temáticos, a mostra apresenta animais taxidermizados, fósseis, minerais, materiais osteológicos, além de conteúdos sobre anatomia humana, citologia e ciências exatas. Com o objetivo de ser um espaço plural para a aprendizagem, a exposição promove a interação com elementos da biodiversidade brasileira e mundial, destacando a riqueza e a interdisciplinaridade da ciência.

Neste cenário, o presente trabalho tem como foco a criação de um aplicativo móvel que visa aprimorar a acessibilidade de pessoas com deficiência visual na Exposição Fritz Muller de Ciências Naturais da FURB (Universidade

Regional de Blumenau – FURB). A proposta envolve o uso de técnicas de Visão Computacional para identificar peças do acervo da exposição, para auxiliar os usuários a receberem descrições detalhadas e interativas por meio de áudio (audiodescrição). A intenção é tornar o ambiente cultural da exposição mais acessível, oferecendo uma experiência mais inclusiva e enriquecedora para pessoas cegas ou com baixa visão.

O objetivo geral deste trabalho é desenvolver um aplicativo móvel capaz de identificar peças do acervo da Exposição Científica Fritz Müller de Ciências Naturais da FURB (Universidade Regional de Blumenau) e realizar a audiodescrição dessas peças para usuários com deficiência visual, promovendo maior acessibilidade e inclusão digital em ambientes museológicos. Para atingir esse propósito, foram definidos os seguintes objetivos específicos: desenvolver o algoritmo de reconhecimento de peças do acervo utilizando uma biblioteca em Python; avaliar a capacidade de identificação dos objetos sob diferentes condições de ambiente, como variações de iluminação e ângulos de captura; implementar o aplicativo móvel na plataforma Unity, com uma interface acessível que integre a biblioteca de reconhecimento desenvolvida; e utilizar a biblioteca RT-Voice para converter em áudio as descrições textuais dos objetos.

2 FUNDAMENTAÇÃO TEÓRICA

Essa seção reúne os principais conceitos e tecnologias que embasam o desenvolvimento do aplicativo assistivo desenvolvido. Inicialmente, aborda-se a inclusão digital e a acessibilidade para pessoas com deficiência visual (subseção 2.1), seguida pela aplicação de redes neurais convolucionais no reconhecimento de objetos (subseção 2.2). Em seguida, apresenta-se o uso do Unity com a biblioteca Barracuda para o desenvolvimento do aplicativo (subseção 2.3) e a integração da síntese de voz por meio do RT-Voice PRO (subseção 2.4). Por fim, são descritos trabalhos correlatos que contribuem com abordagens semelhantes na área de acessibilidade (subseção 2.5).

2.1 INCLUSÃO DIGITAL E ACESSIBILIDADE PARA PESSOAS COM DEFICIÊNCIA VISUAL

A Inclusão é considerada por Azevedo e Barros (2004), um movimento permanente na busca de igualdade de condições e oportunidades para evitar situações de privação em busca de qualidade de vida, desenvolvimento humano, autonomia digital, equidade de oportunidades para indivíduos que encontram-se em situação de desvantagem com relação a outros membros da sociedade. No âmbito digital a inclusão é um direito de acesso ao mundo digital para desenvolvimento intelectual e para o desenvolvimento de capacidade técnica e operacional (Schlünzen, 2005). Essas pessoas precisam muito mais do que somente ter acesso à computadores e celulares, elas precisam ser capazes de operá-los com dependência e autonomia. Nesse caso pessoas com deficiência visual, não basta apenas ter acesso a internet ou a dispositivos eletrônicos, é preciso o desenvolvimento de soluções acessíveis para o uso delas.

A deficiência visual, de acordo com a Organização Mundial de Saúde (OMS, 2023), afeta mais de 2,2 bilhões de pessoas em todo mundo. No Brasil, segundo dados do IBGE 2023, cerca de 6,5 milhões de pessoas apresentam alguma forma de deficiência visual, sendo uma parcela composta por pessoas com falta da visão total, ou baixa visão. Estes dados evidenciam a importância de desenvolvimento de tecnologias para promover acessibilidade para essa grande parcela da população no ambiente digital.

A acessibilidade digital é à capacidade de uma ferramenta, sistema ou conteúdo digital ser utilizado com autonomia por pessoas com deficiência, sem nenhum tipo de barreiras tecnológicas. Esse termo começou a se popularizar na década de 1990, com a maior popularização da internet nos EUA, onde se percebeu a necessidade de promover esse grande acesso universal para pessoas com deficiência também. Retirado de Acessibilidade Brasil, “acessibilidade representa para o nosso usuário não só o direito de acessar a rede de informações, mas também o direito de eliminação de barreiras arquitetônicas, de disponibilidade de comunicação, de acesso físico, de equipamentos e programas adequados de conteúdo e apresentação da informação em formatos alternativos” (Passerirno; Montardo, 2007). É importante destacar que a acessibilidade envolve uma série de práticas, hardware, software e recursos, tais como compatibilidade com leitores de tela (como o NVDA e o JAWS), descrição textual de imagens, uso adequado de cores e contrates, estruturação semântica de páginas e aplicativos, suporte à navegação por teclado e comando de voz, entre outros aspectos (Passerino; Montardo, 2007).

Além da dimensão técnica, a inclusão digital também tem uma forte base legal e ética. A lei Brasileira de Inclusão da Pessoa com Deficiência (lei n 13.146/2015), também conhecida como Estatuto da Pessoa com Deficiência, estabelece, em seu artigo 42, que a acessibilidade em ambientes culturais deve ser garantida, assegurando ás pessoas com deficiência o direito de fruir plenamente das manifestações culturais e artísticas. O uso de tecnologias assistivas e soluções digitais inovadoras é, portanto, uma estratégia concreta para o cumprimento dessa diretriz legal e ética. Além disso no artigo 63, da mesma lei citada anteriormente, se dá como “obrigatório” a acessibilidade nos sítios da internet mantidos por empresas com sede ou representação comercial no País ou por órgãos de governo. Ou seja, a legislação reconhece a importância do acesso à informação e à comunicação e a participação plena de pessoas com deficiência na sociedade como um direito fundamental.

Dessa forma, a inclusão digital de pessoas com deficiência visual vai muito além do acesso à tecnologia, ela precisa do desenvolvimento de soluções que abrangem as reais necessidades desse público, com ferramentas que aumentem sua autonomia, participação e integração com a sociedade (Montardo *et al*., 2008).

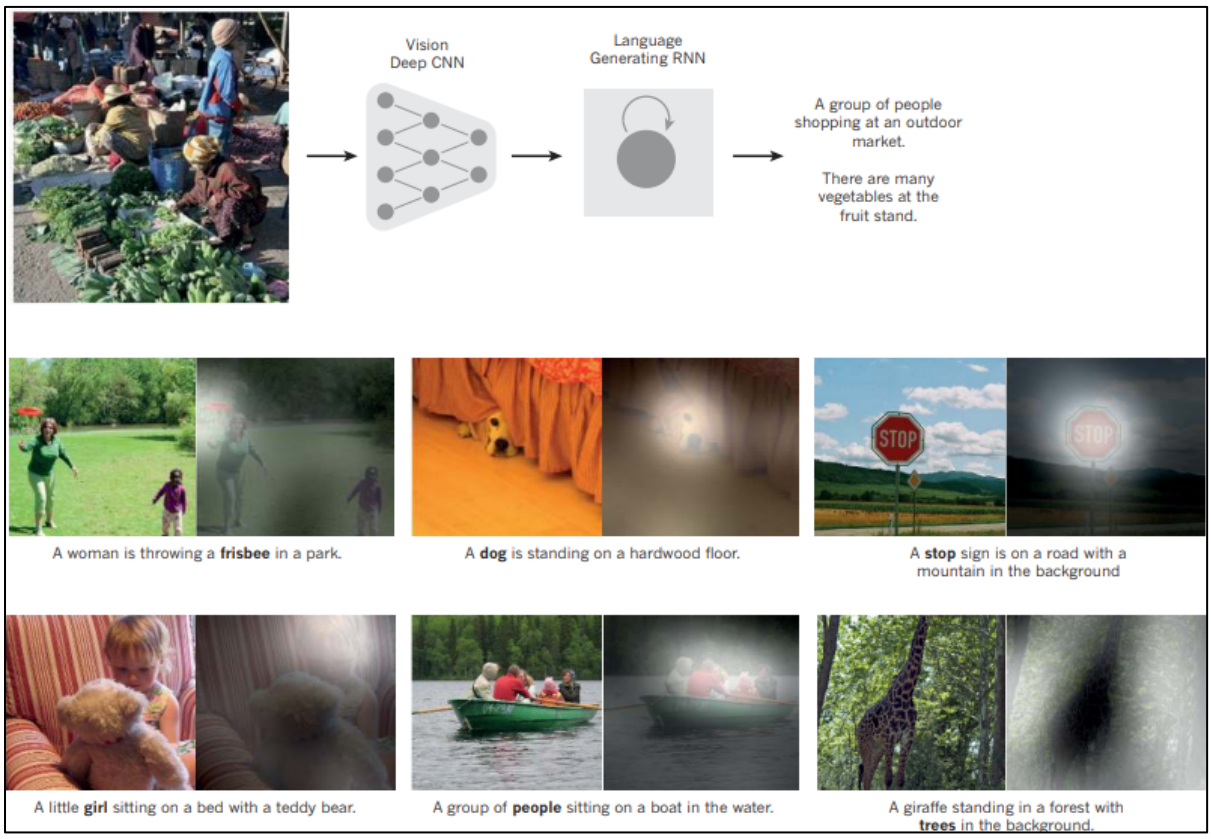
2.2 RECONHECIMENTO DE OBJETOS COM REDES NEURAIS CONVOLUCIONAIS

O reconhecimento de objetos é um ponto crucial dentro da visão computacional, pois envolve a habilidade de identificar, reconhecer e classificar elementos visuais em imagens digitais. Esse processo envolve a identificação de padrões visuais e a associação desses padrões a categorias já conhecidas, sendo utilizado em setores de segurança, automação, monitoramento, medicina, robótica e acessibilidade (Silva, 2018).

Com o progresso da inteligência artificial, principalmente no aprendizado profundo (Deep Learning), as Redes Neurais Convolucionais (CNNs- Convolutional Neural Networks) se tornaram o método principal para tarefas de reconhecimento visual. As CNNs são um tipo de rede neural desenvolvida para tratar dados com estrutura em grade, como imagens se aproveitando de características espaciais e locais através da implementação de filtros convolucionais em diversas camadas da rede (Goodfellow; Bengio; Courville, 2016).

A arquitetura típica de uma CNN consiste em camadas convolucionais encarregadas de extrair características visuais, camadas de *pooling* que fazem a diminuição de dimensões e camadas densas para decisão final. Cada camada adquire cada vez mais representações abstratas da imagem, tornando as CNNs extremamente eficientes na identificação de padrões visuais complexos (Lecun; Bengio; Hintons, 2015). A geração automática de legendas para imagens é uma tarefa que combina Visão Computacional e Processamento de Linguagem Natural, permitindo que sistemas inteligentes descrevam imagens por meio de frases em linguagem natural. Essa abordagem utiliza, geralmente, uma arquitetura composta por duas redes neurais: uma rede neural convolucional (CNN), responsável por extrair características visuais de alto nível da imagem, e uma Rede Neural Recorrente (Recurrent Neural Network - RNN), encarregada de transformar essas representações visuais em sequências textuais coerentes. A inclusão de um mecanismo de atenção (*attention mechanism*) permite que a RNN, ao gerar cada palavra da legenda, concentre-se em diferentes regiões da imagem, conforme a necessidade contextual da descrição. Como ilustrado na Figura 1, esse processo possibilita que o modelo foque dinamicamente em áreas relevantes da imagem (representadas por regiões mais claras) à medida que constrói a frase, o que resulta em descrições mais precisas e contextualizadas (Xu *et al*., 2015).

Figura 1 – Processo de tradução de imagens em texto com CNN e RNN



Fonte: Lecun, Bengio e Hintons (2015).

Modelos como AlexNet (Krizhevsky; Sutskever; Hinton, 2012), que popularizou a utilização de CNNs após sua performance no desafio ImageNet, além de outras arquiteturas como VGGNet, GoogLENet, ResNet e MobileNet

contribuíram para o progresso neste campo. A MobileNet apresentada por Sandler *et al*. (2018), é um a opção que se provou eficaz e leve, destinada a cenários e ambientes com limitações de recursos de computação, como aparelhos móveis.

No âmbito nacional, estudos e pesquisas têm demonstrado o uso bem-sucedido das CNNs em vários cenários. Lima; Melo e Rocha (2020) criaram um sistema de reconhecimento e contagem de moedas brasileiras que combina CNNs com técnicas de pré-processamento em OpenCV, evidenciando que a adaptação do modelo ao contexto local aumenta consideravelmente a precisão. Santos (2018) usou CNNs para identificar objetos em imagens de cenários urbanos, obtendo resultados positivos mesmo em condições diferentes de iluminação. Por outro lado, Oliveira Júnior e Diniz (2024) propuseram uma solução focada na acessibilidade, utilizando redes convolucionais para reconhecer objetos no ambiente e proporcionar feedback visual para indivíduos com deficiência.

O treinamento dessas redes convolucionais exige uma vasta quantidade de dados categorizados, onde imagem dos conjuntos de dados está vinculada a uma classe específica. Métodos como *data augmentation* são frequentemente usados para ampliar a diversidade do conjunto de treinamento, potencializando a habilidade de generalização da rede. No processo de aprendizagem, o modelo modifica seus pesos através de algoritmos de otimização, como gradiente descendente estocástico, minimizando a função de perda entre as previsões e os rótulos reais (Goodfellow; Bengio; Courville, 2016). A identificação de objetos através de CNNs oferece benefícios consideráveis em comparação com os métodos convencionais de classificação por visão computacional. As redes convolucionais aprendem diretamente dos dados, eliminando a necessidade de extrair características manualmente, e exibem resistência a alterações geométricas, omissões e interferências visuais. Isso as torna ideais para usos que necessitam de alta precisão e confiabilidade em cenários reais (Santos, 2018; Silva, 2018).

Além disso, com o aumento da demanda por democratização das tecnologias assistivas, cada vez mais sistemas fundamentados em CNNs estão sendo utilizados para promover a acessibilidade, auxiliando na mobilidade e na interpretação do ambiente por indivíduos com deficiência visual. Esses sistemas usam o reconhecimento visual em tempo real para descrever objetos, textos e barreiras, sendo incorporados a plataformas móveis que suportam a síntese de voz, conforme evidenciado por pesquisas recentes (Oliveira, Junior, Diniz, 2024). Assim, atualmente, as redes neurais convolucionais são o padrão atual em reconhecimento visual, proporcionando uma base tecnológica robusta para a criação de sistemas inteligentes com usos práticos em acessibilidade inclusão digital e interação entre humanos e máquinas.

2.3 DESENVOLVIMENTO DE APLICATIVOS COM UNITY E INTEGRAÇÃO COM BARRACUDA

Com o progresso das tecnologias de Inteligência Artificial (IA) e da computação para dispositivos móveis, o desenvolvimento de aplicativos acessíveis para indivíduos com deficiência visual está se tornando cada dia mais viável e relevante. Neste cenário, a utilização do motor de desenvolvimento Unity, juntamente com a biblioteca Barracuda, possibilita a incorporação de modelos de aprendizado profundo diretamente em aparelhos móveis, permitindo a identificação de objetos em tempo real, sem exigência de conexão com a internet (Santos; Oliveira; Fonseca, 2022).

O Unity é um motor de jogos multiplataforma muito usada em jogos, realidade aumentada, simulações interativas e aplicações educacionais. A sua popularidade é justificada pela simplicidade de uso, ampla documentação e compatibilidade com aparelhos móveis, como smartphones e tablets. Além disso, o Unity disponibiliza funcionalidades para a visão computacional, processamento de imagens, utilização de sensores e integração com bibliotecas externas, tornando-o uma alternativa sólida para projetos que necessitam de interação em tempo real e acessibilidade (Silva; Moura; Cardoso, 2020). Por outro lado, o Barracuda é uma biblioteca de inferência neural criada pela Unity Technologies. Ele possibilita a conversão de modelos treinados em frameworks como Pytorch ou TensorFlow para o formato Open Neural Networks Exchange (ONNX) e a execução nativa do Unity, fazendo uso da CPU, GPU ou, se disponível, da Unidade de Processamento Neural (NPU) do aparelho (Unity Technologies, 2024). Esta integração possibilita a criação de sistemas inteligentes que podem fazer inferências locais, isto é, sem a necessidade de servidores externos, o que é vital em situações em que a latência, a privacidade e a conectividade são elementos vitais, como no caso de aplicativos de acessibilidade.

Por meio da Barracuda, o Unity, utilizando uma rede CNN previamente treinada e convertida para o formato ONNX, é capaz de identificar objetos diretamente a partir da imagem obtida pela câmera do aparelho. O modelo manipula as informações visuais em tempo real e devolve a classificação do objeto identificado. Essa classificação pode ser interpretada pelo aplicativo e transformada em feedback compreensível para usuário, como a emissão de uma mensagem sonora, vibração ou comandos de voz (Pereira; Ribeiro; Nascimento, 2021). Esta estratégia tem sido aplicada em iniciativas acadêmicas e empresariais focadas na inclusão digital, tais como sistemas de navegação assistida, aplicativos de leitura de etiquetas e reconhecimento de ambiente. Em uma pesquisa conduzida por Faria; Santos e Lopes (2022), a aplicação de Inteligência Artificial em aparelhos móveis mostrou-se eficiente na identificação de objetos do dia a dia para indivíduos com deficiência visual, mesmo em locais com variações de luz e ruídos visual. A implementação local do modelo também se revelou benéfica por diminuir o uso de dados móveis e suprimir a demanda por servidores remotos, assegurando maior autonomia e segurança.

Além disso, o Unity suporta bancos de dados locais, como o SQLite que podem ser empregados para guardar dados sobre objetos identificados, histórico de interações e metadados de imagens. Isso possibilita uma experiência do

usuário mais personalizada e ajustada a contextos culturais e individuais, destacando a função da tecnologia como instrumento de inclusão social (Lima; Melo; Rocha, 2020). Assim, a união entre Unity, Barracuda e modelos de redes neurais oferece uma solução técnica eficaz, expansível e ajustável para a criação de aplicativos de assistência. Essa estratégia permite a implementação de técnicas sofisticadas de visão computacional em aparelhos móveis de fácil acesso, auxiliando na promoção da independência e da qualidade de vida de indivíduos com deficiência visual.

2.4 SÍNTESE DE VOZ COM RT-VOICE PRO

A inclusão de pessoa com deficiência visual em aplicações móveis exige o uso de tecnologias que possam transformar informações visuais ou textuais em sinais compreensíveis através de outros sentidos, particularmente a audição. Neste cenário a síntese de voz (*speech synthesis*) é um método frequentemente empregado para converter texto em fala (*text-to-speech*/TTS), promovendo uma interação mais autônoma entre o usuário e a interface digital. Dentre os recursos disponíveis para incorporar TTS em aplicativos criados com Unity, o RT-Voice PRO é um *plugin* comercial que possibilita a incorporação de vozes sintéticas de alta qualidade com uma grande personalização. O RT-Voice PRO é uma ferramenta compatível com o Unity que emprega as vozes nativas do sistema Iwindowa, Android, iOS, MacOS, ou vozes comerciais instaladas localmente para fazer a conversão de texto em tempo real. Isso implica que o software pode produzir falas de forma instantânea, sem a necessidade de conexão com internet ou servidores externos, o que é benéfico em situações de mobilidade urbana ou visitação a locais públicos, onde a conectividade pode ser restrita (Costa; Silva; Andrade, 2022).

O RT-Voice PRO se destaca pela sua habilidade de ajustar a entonação, velocidade e gênero da voz de acordo com o conteúdo a ser expresso ou preferencias do usuário, o que contribui de maneira significativa para a personalização da experiencia de acessibilidade. Ademais, o *plugin* suporta diversos idiomas, facilitando a criação de aplicações em diversos idiomas. A integração do RT-Voice PRO com Unity de maneira nativa ocorre através de *scripts* em C#, que possibilitam a chamada do sistema de TTS diretamente no fluxo lógico da aplicação. Portanto, após a identificação de um objeto através de uma CNN, por exemplo, essa identificação pode ser transformada em uma legenda falada, facilmente compreendida pelo usuário com deficiência visual. O som é gerado localmente, sem qualquer atraso perceptível, tornando a experiencia suave e eficiente (Ferreira; Almeida; Santos, 2021).

Em termos de arquitetura, a utilização do RT-Voice PRO complementa os módulos de visão computacional e inferência neural, atuando como uma última cama na comunicação entre o homem e a máquina. Este método está em conformidade com a ideia de assistência sensorial tecnológica, que procura substituir ou complementar sentidos comprometidos através dessas ferramentas computacionais, com o objetivo de fomentar a independência e a inclusão digital (Moura; Cardoso; Gomes, 2020). Estudos recentes apontam que soluções baseadas em TTS são amplamente aceitas por indivíduos com deficiência visual, especialmente quando combinadas com feedbacks ágeis, voz natural e baixo uso de bateria do aparelho (Barros; Nascimento; Paula, 2022). Assim, a implementação de ferramentas como o RT-Voice PRO é uma opção eficaz e pertinente quando conciliadas com o avanço das tecnologias com os preceitos do design universal e da acessibilidade.

2.5 TRABALHOS CORRELATOS

Nesta subseção são apresentados trabalhos que se correlacionam em objetivos e características com o estudo proposto. O primeiro trabalho (Quadro 1) é um protótipo de jogo acessível para pessoas com deficiência visual, desenvolvido por Batistel (2020), utilizando Unity, C# e RT-Voice Pro. O segundo trabalho (Quadro 2), de Ferreira; Almeida e Santos (2021), consiste em um aplicativo em Python para reconhecimento de cédulas monetárias brasileiras voltado a deficientes visuais, utilizando técnicas de processamento de imagens e classificadores baseados em Máquina de Vetores de Suporte (Support Vector Machine - SVM). O terceiro trabalho (Quadro 3), de Colodette (2023), apresenta um aplicativo móvel para leitura assistida por meio de Reconhecimento Óptico de Caracteres (Optical Character Recognition – OCR) e conversão de texto em áudio, desenvolvido em Flutter e Python, com uso de TensorFlow e Flask no backend.

Quadro 1 – Estudo de Tecnologias Assistivas para Deficientes Visuais Aplicadas ao Jogo Furbot

Referência	Batistel (2020)
Objetivos	Desenvolver uma versão acessível do jogo Furbot para permitir que pessoas com deficiência visual joguem de forma independente, utilizando recursos como audiodescrição e *text-to-speech* (TTS).
Principais funcionalidades	Funcionalidades incluem audiodescrição, *text-to-speech*, sensores de ambiente com feedback sonoro e um sistema de *checkpoints* para orientação e acessibilidade no jogo Furbot.
Ferramentas de desenvolvimento	Unity, linguagem de programação C# e o RT-Voice Pro e uma ferramenta de *text-to-speech* (TTS).
Resultados e conclusões	O protótipo acessível do Furbot foi eficaz para usuários com deficiência visual, atendendo aos objetivos de inclusão e acessibilidade, embora tenha sido recomendado para melhorar o TTS e a navegação.

Fonte: elaborado pelo autor.

Quadro 2 – Aplicativo capaz de reconhecer cédulas monetárias para deficientes visuais

Referência	Ferreira; Almeida e Santos (2021)

Objetivos	Facilitar a vida de uma pessoa com deficiência visual, permitindo-lhe usar um aplicativo apto para reconhecer o valor de uma cédula monetária e informá-lo ao deficiente visual.
Principais funcionalidades	Função de identificar cédulas da moeda brasileira e Processamento de Imagens.
Ferramentas de desenvolvimento	Python, equalização de histograma, canny, limiarização, filtro Bilateral e entre outras.
Resultados e conclusões	Dos 39 classificadores testados, os dois com maior acurácia (90,33%) foram baseados em SVM. A melhor configuração foi a 1, com uma média de 89,28% nas três métricas. Um *grid search* foi realizado para otimizar os parâmetros, testando cinco valores diferentes para cada um.

Fonte: elaborado pelo autor.

Quadro 3 – Aplicativo Móvel de Assistência à Leitura para Deficientes Visuais através de VC e OCR

Referência	Colodette (2023)
Objetivos	Criar um aplicativo móvel que converte textos em áudio para melhorar o acesso à informação para pessoas com deficiência visual, utilizando tecnologias de OCR e síntese de voz.
Principais funcionalidades	Captura de imagens com texto, reconhecimento de caracteres via OCR e conversão do texto em áudio para facilitar a compreensão e navegação.
Ferramentas de desenvolvimento	Desenvolvido com Flutter, Python, Flask, TensorFlow (para implementar OCR) e visão computacional.
Resultados e conclusões	O aplicativo mostrou-se eficaz na assistência à leitura para deficientes visuais, facilitando o acesso à informação e promovendo inclusão social, com potencial para futuras melhorias.

Fonte: elaborado pelo autor.

3 DESCRIÇÃO

Nessa seção será apresentado o processo de desenvolvimento do aplicativo voltado à identificação de objetos do acervo da Fundação Universidade Regional de Blumenau (FURB), com foco na acessibilidade para pessoas com deficiência visual. Serão descritas as etapas que envolvem o treinamento do modelo de aprendizado profundo em Python, utilizando a arquitetura MobileNetV2, até a integração desse modelo no ambiente Unity com suporte para inferência em tempo real via Barracuda. Além disso, será abordada a implementação do sistema de feedback auditivo com o uso da ferramenta RT-Voice Pro, que transforma os resultados de reconhecimento em saída por voz. Com base nesses objetivos, foram definidos os Requisitos Funcionais (RFs) e os Requisitos Não Funcionais (RNFs), detalhados no Quadro 4 e Quadro 5, respectivamente.

Quadro 4 – Requisitos Funcionais

O sistema deve ser capaz de utilizar a câmera do dispositivo móvel para capturar imagens em tempo real.
O sistema deve identificar objetos do acervo com base em um modelo treinado.
O sistema deve anunciar em voz alta o nome ou a descrição do objeto identificado utilizando o RT-Voice Pro.
O sistema deve ser capaz de realizar todo o processo de reconhecimento e resposta por voz sem depender de conexão com a internet.
O sistema deve apresentar uma interface adaptada para pessoas com deficiência visual, com interação mínima e intuitiva.

Fonte: elaborado pelo autor.

Quadro 5 – Requisitos Não Funcionais

O sistema deve realizar o reconhecimento e fornecer o feedback por voz em até 3 segundos após o apontamento da câmera.
O sistema deve ser executável em dispositivos Android, especialmente versões mais recentes.
O sistema deve seguir princípios de acessibilidade, como contraste, leitura automática e ausência de elementos visuais excessivos.
O sistema não deve apresentar falhas ao identificar objetos, mesmo com variações de iluminação ou ângulos.
O modelo ONNX, os dados de suporte e o sistema devem ocupar espaço reduzido no armazenamento do dispositivo, visando bom desempenho.

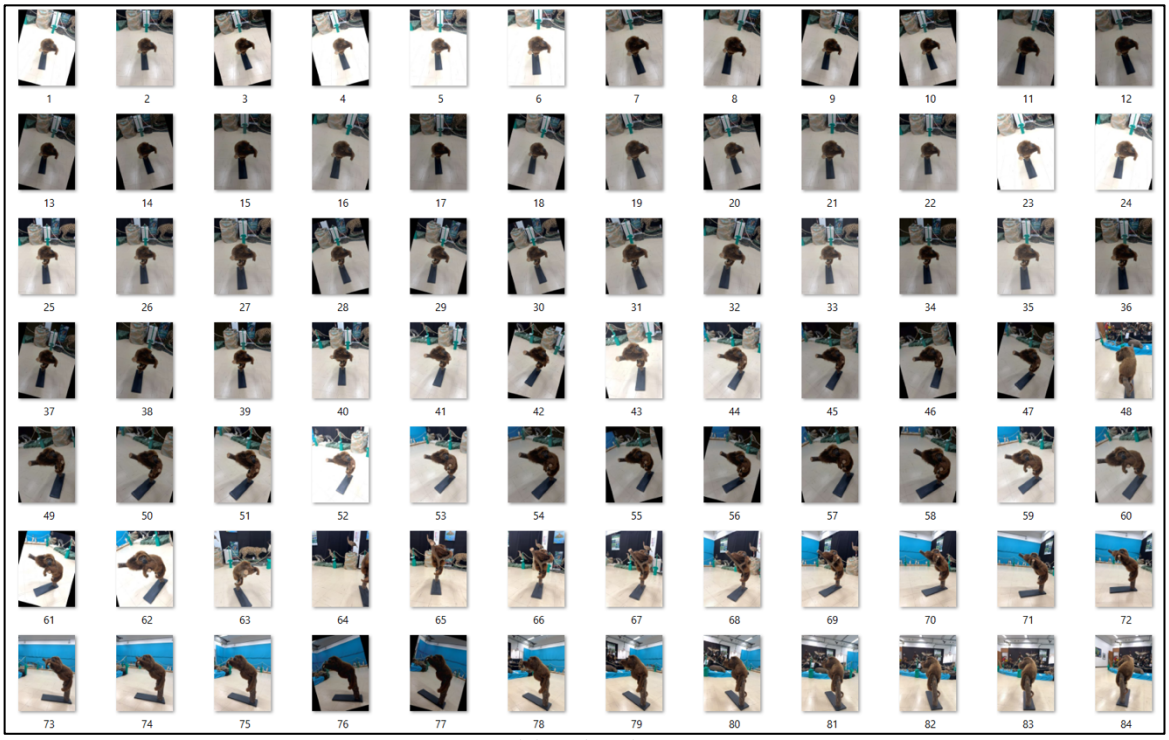
Fonte: elaborado pelo autor.

Para a construção da aplicação de reconhecimento de objetos voltada à inclusão de pessoas com deficiência visual, utilizou-se a linguagem de programação Python durante a etapa de treinamento do modelo de aprendizado profundo, aproveitando sua vasta gama de bibliotecas e facilidade de manipulação de dados. O modelo de rede neural escolhido foi o MobileNetV2, devido à sua leveza e alta precisão para dispositivos móveis, possibilitando a inferência em tempo real com baixo custo computacional.

A Figura 2 apresenta um exemplo representativo das imagens empregadas no processo de treinamento do modelo. As imagens utilizadas no treinamento foram capturadas diretamente com a câmera do celular e, posteriormente,

processadas por meio de um *script* desenvolvido para gerar variações em diferentes rotações e condições de iluminação como mostra na Figura 3. Esse procedimento teve como objetivo ampliar a diversidade da base de dados, melhorando a robustez do modelo frente a variações reais no ambiente. Ao todo, foram utilizadas 541 imagens, sendo 389 destinadas ao conjunto de treino e 152 ao conjunto de validação, devidamente separadas por suas respectivas classes. O Fluxo de treinamento do modelo de reconhecimento de objetos com MobileNetV2 pode ser visto na Figura 4.

Figura 2 – Exemplo das imagens usadas para o treinamento



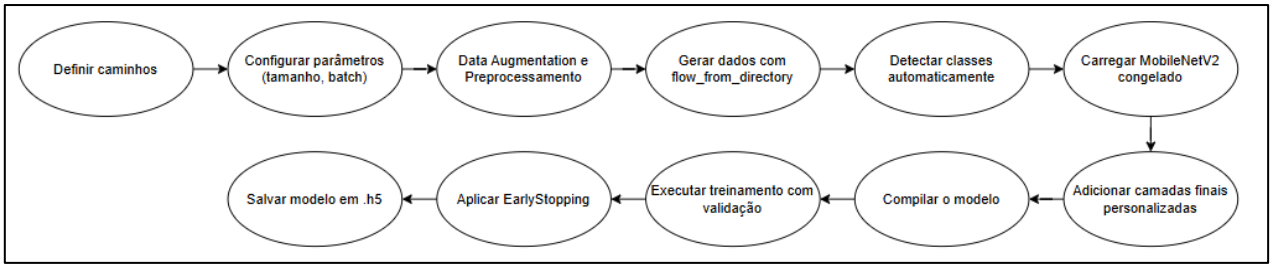
Fonte: elaborado pelo autor.

Figura 3 – Script para criar imagens

```
def ajustar_brilho_contraste(img, brilho=0, contraste=1.0):
    return cv2.convertScaleAbs(img, alpha=contraste, beta=brilho)

def rotacionar_img(img, angulo):
    (h, w) = img.shape[:2]
    centro = (w // 2, h // 2)
    matriz = cv2.getRotationMatrix2D(centro, angulo, 1.0)
    return cv2.warpAffine(img, matriz, (w, h))

def aplicar_zoom(img, fator):
    if fator == 1.0:
        return img
    h, w = img.shape[:2]
    novo_h, novo_w = int(h / fator), int(w / fator)
    start_y = (h - novo_h) // 2
    start_x = (w - novo_w) // 2
    cropped = img[start_y:start_y + novo_h, start_x:start_x + novo_w]
    return cv2.resize(cropped, (w, h))

imagens = [f for f in os.listdir(input_folder) if f.lower().endswith((".jpg",
".jpeg", ".png"))]
```

Fonte: elaborado pelo autor.

Figura 4 – Fluxo de treinamento do modelo de reconhecimento de objetos com MobileNetV2



Fonte: elaborado pelo autor.

Inicialmente, foram definidos os diretórios contendo as imagens organizadas em subpastas para cada classe. Além disso, parâmetros como tamanho da imagem (224x224 pixels), tamanho do lote (*batch size*) e o caminho de saída do modelo treinado foram configurados (Figura 5). A escolha do tamanho da imagem foi baseada na arquitetura da MobileNetV2, que requer entradas com essa dimensão.

Figura 5 – Código fonte configurando o modelo

```
# Caminhos
base_dir = r'C:\Users\Leonardo Fernandes\Pictures\Imagens Objeto
TCC\Treino'
train_dir = os.path.join(base_dir, 'train')
val_dir = os.path.join(base_dir, 'val')
output_model = r'C:\Users\Leonardo
Fernandes\ExpoFritz\Assets\StreamingAssets\model_mobilenet.h5'

# Parâmetros
image_size = (224, 224)  # MobileNetV2 espera 224x224
batch_size = 32
```

Fonte: elaborado pelo autor.

Para aumentar a robustez do modelo e evitar o *overfitting*, foi aplicado *data augmentation* nas imagens de treino. Técnicas como rotação, variação de brilho, deslocamento, zoom e espelhamento horizontal foram utilizadas. Já para o conjunto de validação, foi aplicado apenas o reescalonamento dos valores de pixel para a faixa entre 0 e 1. O uso do `ImageDataGenerator` permitiu a geração dinâmica de lotes de imagens durante o treinamento. As imagens foram organizadas em duas pastas principais: `train` e `val`. O gerador de dados carrega automaticamente as imagens dessas pastas e as classifica com base na estrutura dos diretórios. O número total de classes é detectado dinamicamente a partir das subpastas presentes no diretório de treinamento.

O modelo foi compilado com o otimizador Adam, uma taxa de aprendizado de 1e-4, e a função de perda `categorical_crossentropy`, ideal para tarefas de classificação multiclasse. A métrica de avaliação escolhida foi a acurácia, que permite acompanhar o desempenho durante o treinamento e validação. O modelo foi treinado por até 15 épocas, com uso da técnica de Early Stopping, que interrompe o treinamento caso a métrica de validação pare de melhorar após 5 épocas consecutivas. Essa abordagem evita "sobreajuste" (*overfitting*) e reduz o tempo total de treinamento. Ao final do processo, o modelo ajustado foi salvo em formato `.h5` e exportado para o diretório utilizado no projeto Unity.

Após o treinamento, o modelo foi convertido para o formato Open Neural Network Exchange (ONNX) como mostra na Figura 6, permitindo sua integração ao ambiente Unity com o uso da biblioteca Barracuda, que viabiliza a execução de redes neurais diretamente no dispositivo, sem a necessidade de conexão com servidores externos. Essa abordagem garante maior responsividade e funcionamento offline, fundamental para acessibilidade em ambientes sem acesso à internet.

Figura 6 – Código fonte do conversor de arquivo do tipo `.H5` para `.ONNX`

```
import tensorflow as tf
import tf2onnx

# Caminho do seu modelo .h5
model = tf.keras.models.load_model(r"C:\Users\Leonardo
Fernandes\ExpoFritz\Assets\StreamingAssets\model_mobilenet.h5")

# Exportar para ONNX
spec = (tf.TensorSpec((None, 224, 224, 3), tf.float32, name="input"),)
output_path = "model_mobilenet.onnx"
model_proto, _ = tf2onnx.convert.from_keras(model, input_signature=spec,
output_path=output_path)

print(f"Modelo convertido para: {output_path}")
```

Fonte: elaborado pelo autor.

Primeiramente (Figura 6), a biblioteca TensorFlow é utilizada para carregar o modelo com a função `load_model()`, apontando para o caminho onde o arquivo está armazenado. Em seguida, define-se uma assinatura de entrada (`input_signature`) por meio de um `TensorSpec`, que especifica a forma dos dados de entrada esperados pela rede — neste caso, imagens com dimensões de 224x224 pixels e 3 canais de cor (RGB), compatíveis com a arquitetura MobileNetV2. A conversão é feita com a função `from_keras()` da biblioteca `tf2onnx`, que transforma o modelo Keras em um gráfico computacional compatível com o formato ONNX. Por fim, o modelo convertido é salvo no caminho especificado, ficando pronto para ser importado no Unity e utilizado com a Barracuda para inferência em tempo real.

Após a conversão do modelo treinado no formato `.h5` para o formato ONNX, a próxima etapa consistiu em incorporá-lo ao ambiente de desenvolvimento Unity (Figura 7). Para isso, foi utilizada a biblioteca Barracuda, uma solução desenvolvida pela própria Unity para permitir a execução local de redes neurais treinadas em frameworks como TensorFlow. A escolha por essa abordagem foi motivada pela necessidade de garantir execução em tempo real e funcionamento offline, especialmente em ambientes como museus, onde a conexão com a internet pode ser limitada ou inexistente. O arquivo `.ONNX` foi colocado na pasta `Modelos` dentro da pasta `Assets`, que é uma pasta especial no Unity usada para armazenar arquivos que precisam ser acessados em tempo de execução, sem que sejam alterados durante a *build*. Em seguida, um script em C# foi desenvolvido utilizando a API da Barracuda, sendo responsável por carregar o modelo, preparar os dados de entrada e executar a inferência.

Figura 7 – Código fonte carregando o modelo ONNX

```
// Importações relacionadas ao Barracuda (ONNX e inferência)
using Unity.Barracuda;

public NNModel modelAsset;

private Model runtimeModel;
private IWorker worker;

void Start()
{
    // Carregar modelo ONNX no formato interno do Barracuda
    runtimeModel = ModelLoader.Load(modelAsset);
    worker = WorkerFactory.CreateWorker(WorkerFactory.Type.Auto,
runtimeModel);
```

Fonte: elaborado pelo autor.

Durante a execução, o aplicativo acessa a câmera do dispositivo móvel para capturar imagens em tempo real. Essas imagens passam por um processo de pré-processamento que inclui redimensionamento para 224x224 pixels, conversão para tensor e normalização dos valores de pixel para se adequar ao padrão de entrada exigido pela MobileNetV2. O tensor gerado é então enviado para o modelo, que realiza a predição e retorna uma classificação correspondente ao objeto detectado na imagem. O resultado da predição é interpretado no próprio Unity, permitindo que o aplicativo exiba o nome do objeto reconhecido em tempo real na interface do usuário. Essa arquitetura baseada em inferência local garante alta performance e responsividade, e é essencial para manter a acessibilidade e a experiência fluida para usuários com deficiência visual. O código fonte utilizando o Barracuda para importar imagens

Figura 8 – Código fonte utilizando o Barracuda para importar imagens

```
IEnumerator RunInference()
    {
        // Captura frame atual
        Texture2D photo = new Texture2D(camTexture.width,
camTexture.height, TextureFormat.RGB24, false);
        photo.SetPixels(camTexture.GetPixels());
        photo.Apply();

        // Redimensiona para 224x224
        Texture2D resized = ResizeTexture(photo, 224, 224);
        Destroy(photo);

        // Prepara dados para o tensor
        float[] floatValues = new float[224 * 224 * 3];
        Color32[] pixels = resized.GetPixels32();
        Destroy(resized);

        for (int i = 0; i < pixels.Length; i++)
        {
            floatValues[i * 3 + 0] = pixels[i].r / 255.0f;
            floatValues[i * 3 + 1] = pixels[i].g / 255.0f;
            floatValues[i * 3 + 2] = pixels[i].b / 255.0f;
        }

        // Cria tensor [1, 224, 224, 3]
        Tensor input = new Tensor(1, 224, 224, 3, floatValues);

        // Executa inferência
        worker.Execute(input);
        Tensor output = worker.PeekOutput();

        // Analisa saída
        int maxIndex = ArgMax(output.ToReadOnlyArray());
        float confidence = output[maxIndex];

        if (confidence < 0.5f)
        {
            notificationText.color = Color.red;
            notificationText.text = "Nada detectado";
        }
        else
        {
            string detectedLabel = classNames[maxIndex];
            notificationText.color = Color.green;
            notificationText.text = $"Detectado: {detectedLabel}
({(confidence * 100):F2}%)";

            SpeakLabel(detectedLabel);
            Handheld.Vibrate();
        }

        input.Dispose();
        output.Dispose();

        yield return null;
    }
```

Fonte: elaborado pelo autor.

A aplicação também incorpora o RT-Voice Pro, uma ferramenta avançada de Text-To-Speech (TTS) desenvolvida para o Unity. Essa solução permite a conversão dinâmica de qualquer texto em fala, utilizando as vozes nativas do sistema operacional, seja no Android, iOS, Windows ou MacOS, sem a necessidade de conexão com a internet. A integração do RT-Voice Pro com o aplicativo (Figura 9) garante um feedback auditivo imediato, fundamental para a acessibilidade de usuários com deficiência visual.

A Figura 9 apresenta o método `SpeakLabel(string label)` que é responsável por converter o rótulo reconhecido pela rede neural em uma frase audível, utilizando o RT-Voice. A configuração do RT-Voice no projeto é simples, e após a importação do pacote do RT-Voice para o Unity, é possível utilizar comandos como `Speaker.Instance.Speak("mensagem")` para iniciar a reprodução de qualquer mensagem de forma síncrona ou assíncrona. O desenvolvedor pode ainda definir a voz, idioma, velocidade, volume e *pitch* da fala, bem como monitorar se há alguma fala em execução por meio da propriedade `Speaker.Instance.isSpeaking`. A escolha da frase é feita de forma condicional com base no objeto identificado, garantindo um feedback verbal específico para cada item.

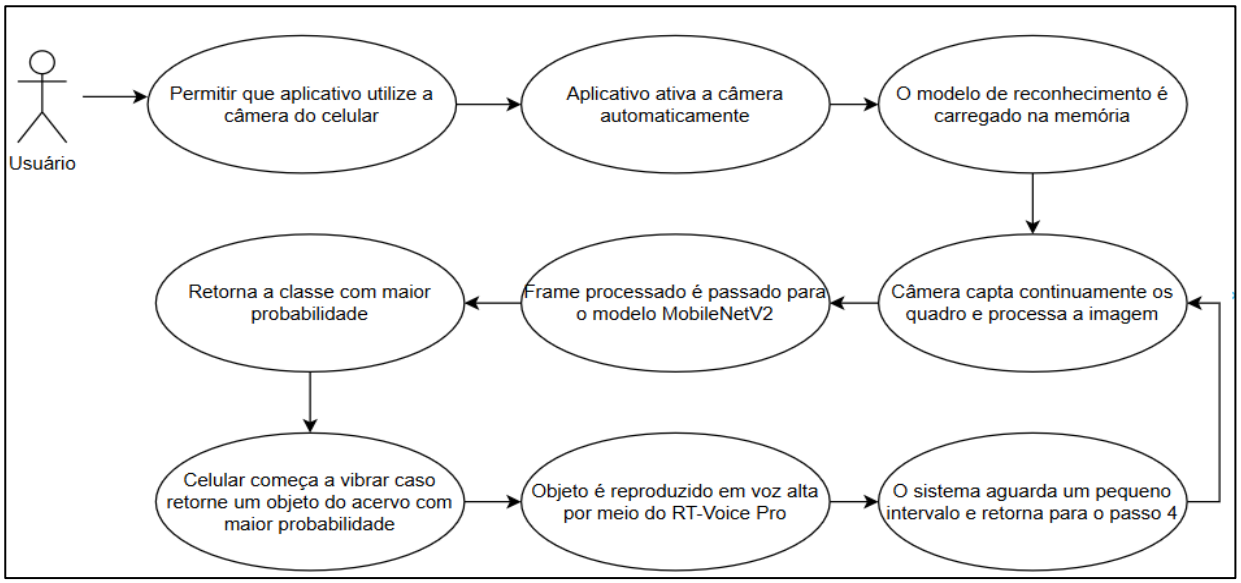
Figura 9 – Código fonte explicando o RT-Voice Pro

```
void SpeakLabel(string label)
{
    string phrase = label switch
    {
        "joystick" => "Isso é um Joystick",
        "bugio" => GetBugioDescricao(),
        "chimpanze" => GetChimpanzeDescricao(),
        _ => $"Detectei {label}"
    };

    if (!Speaker.Instance.isSpeaking)
    {
        Speaker.Instance.Speak(phrase);
    }
}
```

Fonte: elaborado pelo autor.

No contexto do aplicativo, o nome do objeto reconhecido é transformado em uma frase significativa e pronunciado automaticamente logo após a inferência da rede neural, complementando o feedback visual (exibição em tela) e tátil (vibração do dispositivo). Com essa infraestrutura, o sistema desenvolvido possibilita o reconhecimento rápido de itens do acervo da FURB, promovendo inclusão digital por meio da tecnologia e da inteligência artificial. A Figura 10 apresenta o fluxo de funcionamento do aplicativo móvel desenvolvido, que utiliza técnicas de visão computacional e redes neurais convolucionais para identificar objetos em tempo real com base em imagens capturadas pela câmera do dispositivo. O processo se inicia com a permissão do usuário para acesso à câmera do celular. Após concedida a permissão, o aplicativo ativa automaticamente a câmera e carrega em memória o modelo de reconhecimento treinado com MobileNetV2, previamente convertido para o formato ONNX e integrado ao Unity por meio da biblioteca Barracuda.

Figura 10 – Fluxo de uso de usuário no aplicativo ExpoFritz



Fonte: elaborado pelo autor.

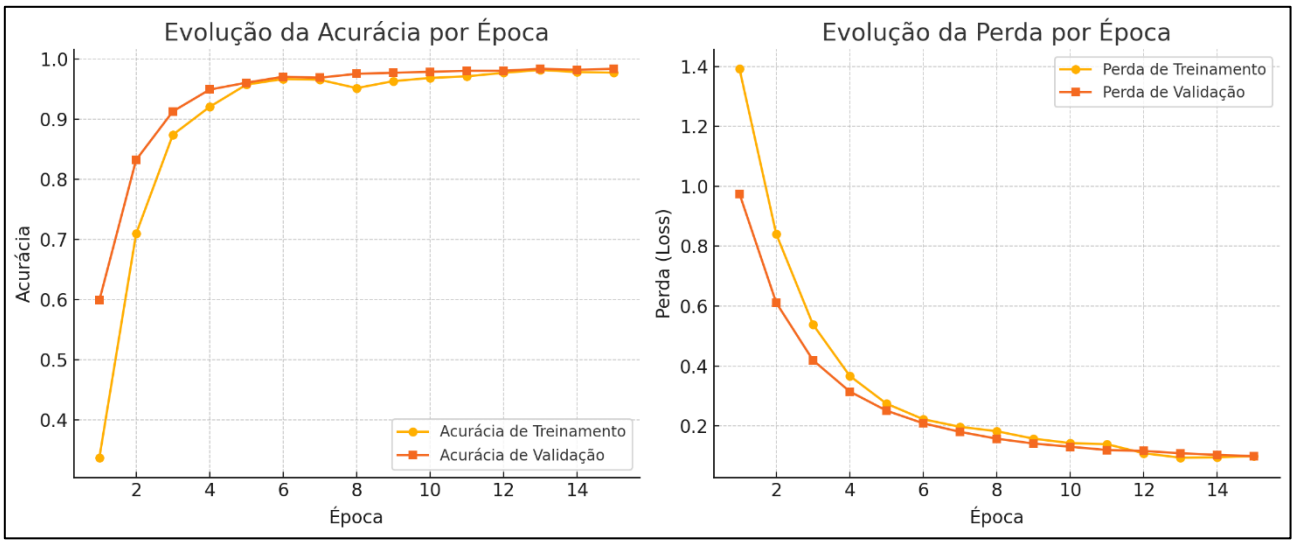
Com a câmera em funcionamento, quadros (*frames*) são continuamente capturados e processados. A cada intervalo de tempo definido, uma imagem redimensionada e passada como entrada para o modelo MobileNetV2. A rede neural realiza a inferência e retorna a classe com maior probabilidade, ou seja, o objeto mais provável presente na imagem analisada. Caso o objeto detectado pertença ao acervo previamente treinado, o sistema executa ações de acessibilidade: o celular vibra para fornecer um feedback tátil ao usuário e o nome do objeto é reproduzido em voz alta por meio da biblioteca RT-Voice Pro. Em seguida, o sistema aguarda um pequeno intervalo antes de reiniciar o ciclo de captura e análise da próxima imagem, mantendo assim o reconhecimento contínuo em tempo real e de forma responsiva.

4 RESULTADOS

Nesta seção, são apresentados e discutidos os principais resultados obtidos ao longo do desenvolvimento da aplicação móvel voltada à inclusão de pessoas com deficiência visual. O foco principal está na avaliação do desempenho do modelo de rede neural treinado com MobileNetV2, sua integração ao ambiente Unity por meio do formato ONNX e da biblioteca Barracuda, e a experiência final de uso do aplicativo com os recursos de acessibilidade habilitados, como feedback por voz e vibração. São descritas as taxas de acerto do modelo, os objetos corretamente reconhecidos em diferentes condições de luminosidade e ângulo, além da resposta em tempo real do sistema. Também são avaliados aspectos de usabilidade e acessibilidade da aplicação, com base em testes práticos realizados em ambientes semelhantes ao público-alvo, como museus e exposições. Os resultados demonstram a viabilidade técnica da solução e sua capacidade de fornecer suporte acessível, funcional e eficiente para o reconhecimento e a descrição de objetos do acervo.

Como mostrado na Figura 11, durante o processo de treinamento do modelo MobileNetV2, foi possível observar uma evolução significativa nos indicadores de desempenho ao longo das 15 épocas executadas. A acurácia de treinamento aumentou de forma consistente desde a primeira época, alcançando valores superiores a 97% nas últimas iterações. De forma semelhante, a acurácia de validação também apresentou crescimento progressivo, estabilizando-se acima de 98%, o que evidencia a capacidade do modelo de generalizar bem para dados não vistos durante o treinamento.

Figura 11 – Evolução da acurácia e da perda por época do modelo MobileNetV2



Fonte: elaborado pelo autor.

Paralelamente, a função de perda (*loss*) demonstrou uma redução contínua tanto no conjunto de treinamento quanto no de validação. Essa queda constante indica que o modelo foi capaz de aprender os padrões relevantes dos dados sem apresentar sinais evidentes de "sobreajuste" (*overfitting*). Os resultados obtidos (Figura 11) confirmam a eficácia do processo de treinamento e a robustez da arquitetura utilizada para a tarefa de reconhecimento de objetos.

Com o objetivo de avaliar o desempenho do modelo em situações reais de uso, foram realizados testes em diferentes dispositivos móveis. Os aparelhos utilizados foram: Redmi Note 8, Redmi Note 12 e Samsung S23 Ultra. Os testes buscaram verificar o funcionamento da inferência em tempo real, a fluidez da aplicação, o tempo de resposta do reconhecimento de objetos, bem como a efetividade do feedback auditivo fornecido pela biblioteca RT-Voice. A Figura 12 apresenta a interface do aplicativo em uma situação real de uso. A interface é propositalmente simples, considerando o público-alvo composto por pessoas com deficiência visual. No entanto, o rótulo com o texto na tela foi mantido para evidenciar que o objeto foi corretamente reconhecido, conferindo veracidade aos resultados.

Figura 12 – Resultados retirados do vídeo dos resultados

Fonte: elaborado pelo autor.

O Quadro 6 apresenta os resultados médios de acurácia do modelo nos testes realizados com os três dispositivos utilizados: Redmi Note 8, Redmi Note 12 e Samsung S23 Ultra. Observa-se que, embora o Samsung S23 Ultra tenha apresentado uma acurácia média ligeiramente inferior na classe "Chimpanzé" em comparação aos demais (85,42%), esse resultado deve ser analisado com cautela. O aparelho da Samsung demonstrou capacidade superior de reconhecimento em ambientes mais complexos, onde os demais dispositivos apresentaram dificuldades ou falhas completas de inferência.

Quadro 6 – Comparativos de resultados dos aparelhos.

Celulares	Chimpanzé (Média de acurácia)	Bugio (Média de acurácia)
Redmi Note 8	87,98%	73,26%
Redmi Note 12	88,89%	77,44%
Samsung S23 Ultra	85,42%	86,70%

Fonte: elaborado pelo autor.

O Redmi Note 8, por exemplo, não conseguiu realizar a detecção em diversos desses cenários, e o Redmi Note 12 apresentou reconhecimento inconsistente. Já o Samsung S23 Ultra manteve um desempenho robusto mesmo sob condições adversas, com média superior a 90% de acurácia em situações em que todos os dispositivos conseguiam realizar a inferência. Esses resultados indicam que, apesar da variação estatística pontual, o modelo apresentou maior precisão e confiabilidade quando executado em um dispositivo com maior capacidade de processamento.

Durante os testes realizados com os três dispositivos, observou-se um comportamento recorrente de encerramento inesperado do aplicativo (*crash*), ocasionado pelo esgotamento da memória RAM. Esse problema foi identificado de forma consistente em todos os aparelhos, especialmente durante períodos prolongados de uso com a inferência em tempo real ativada. O consumo elevado de recursos ocorre devido à combinação do uso da câmera, da execução contínua da rede neural para classificação das imagens e da síntese de voz com a biblioteca RT-Voice. Como consequência, a memória não era liberada de forma eficiente, o que gerava o fechamento forçado da aplicação pelo sistema operacional do dispositivo. Esses *crashes* foram registrados em vídeo durante os testes práticos e estão disponíveis no repositório do projeto como evidência do comportamento observado. O log apresentado na Figura 13 evidencia falhas no gerenciamento da superfície de vídeo e indica a quebra da comunicação com o pipeline de captura da câmera, o que sugere uma saturação dos recursos do sistema, levando à finalização forçada do processo.

Figura 13 – Log lotação de RAM

```
Error Surface queueBuffer: error queueing buffer to SurfaceTexture,
-32
Error Camera3-OutputStream returnBufferCheckedLocked: Stream 0:
Error queueing buffer to native window: Broken pipe (-32)
QCamera <HAL><ERROR> acquirePerfLock: Failed to acquire the perf
lock
QCamera <HAL><ERROR> stop: Attempt to stop inactive channel
mm-camera <MCT><ERROR> stop_sof_check_thread: Returning as SOF timer
thread not yet initialized
Fatal signal 6 (SIGABRT), code -1 (SI_QUEUE) in tid 17031
(.unity.SampleApp), pid 16980 (.unity.SampleApp)
```

Fonte: elaborado pelo autor.

5 CONCLUSÕES

O propósito desse estudo foi o desenvolvimento de um aplicativo móvel acessível, para identificar objetos em ambientes como exposições, com foco na inclusão de pessoas com deficiência visual. Com a construção de uma solução funcional que combina visão computacional, inferência em tempo real com redes neurais convolucionais e recursos de acessibilidade, como síntese de voz e feedback por vibração, sendo que essas propostas foram atendidas. A combinação das ferramentas Unity como motor gráfico, Barracuda para executar o modelo treinado em ONNX, RT-Voice Pro para converter texto em fala mostrou-se apropriada, possibilitando o desenvolvimento de uma aplicação leve, responsiva e apta a operar offline, mesmo em aparelhos móveis de hardware mais básico, como evidenciado nos testes realizados com o smartphone Redmi Note 8.

As conclusões obtidas, tanto no processo de treinamento quanto na avaliação prática do aplicativo, apontaram elevadas taxas de acerto e uma boa generalização do modelo MobileNetV2. O uso de uma base de dados formada por imagens reais e posteriormente ampliadas em diversas condições de luz, rotação e angulos contribuiu para a melhoria da qualidade da rede. A utilização foi satisfatória, com uma resposta apropriada, mesmo em aparelhos de entrada. O trabalho apresentou como principais contribuições uma solução viável e acessível para promover a inclusão em ambientes culturais, possibilitando que indivíduos com deficiência visual tenham uma experiência mais informativa e autônoma através do reconhecimento automático de objetos. Além disso, a metodologia usada auxilia no progresso do emprego de redes neurais embarcadas e na criação de aplicações inclusivas que incorporam funcionalidades nativas de inteligência artificial.

Entre as limitações identificadas, destaca-se a dependência do sistema de câmera do Android, que pode apresentar instabilidades em determinados dispositivos e versões do sistema, além da necessidade de ampliar o banco de dados com mais objetos para melhorar a escalabilidade da aplicação. Neste contexto, levantam-se algumas possíveis melhorias e extensões para expandir o trabalho realizado, sendo elas:

a) a otimização da performance geral do aplicativo, com foco especial na gestão de memória e na redução do tempo de resposta do feedback auditivo, buscando evitar falhas e encerramentos inesperados em dispositivos com menor capacidade de hardware, além de tornar a comunicação por voz mais imediata e eficiente durante o uso;
b) a ampliação da base de dados com novas classes de objetos do museu, com o objetivo de tornar o aplicativo aplicável a diferentes exposições e instituições culturais;
c) a integração com comandos de voz, possibilitando que o usuário interaja com o aplicativo por meio de perguntas ou solicitações, tornando a navegação ainda mais acessível;
d) o suporte à leitura de QR Codes ou etiquetas NFC associadas a objetos, oferecendo uma alternativa de reconhecimento baseada em códigos físicos que complementa a visão computacional;
e) a adaptação da aplicação para múltiplos idiomas, permitindo que usuários estrangeiros também possam utilizar o aplicativo em visitas internacionais;
f) a adição de um modo “audiodescrição contínua”, em que o sistema descreve o ambiente ao redor conforme o usuário se movimenta, identificando e narrando objetos automaticamente com base em sua posição na câmera;
g) a criação de um painel de administração para museus, permitindo cadastrar, editar e gerenciar as imagens e descrições dos objetos diretamente de uma interface web sincronizada com o aplicativo;
h) a realização de testes com usuários reais com deficiência visual, incluindo entrevistas e observações de uso, para validar e aprimorar a experiência de usabilidade e acessibilidade de forma empírica.

REFERÊNCIAS

AZEVEDO, P. H.; BARROS, J. F. **O nível de participação do Estado na gestão do esporte brasileiro como fator de inclusão social de pessoas portadoras de deficiência.** *Revista Brasileira de Ciência e Movimento*, Brasília, v. 12, n. 1, p. 77–84, jan./mar. 2004.

BARROS, Lúcia M.; NASCIMENTO, Paulo A.; PAULA, José R. **Análise da percepção de usuários cegos sobre sistemas de leitura automática de tela com voz sintetizada.** *Revista Tecnologia e Sociedade*, v. 18, n. 2, p. 144–160, 2022.

BATISTEL, Caroline. **ESTUDO DE TECNOLOGIAS ASSISTIVAS PARA DEFICIENTES VISUAIS APLICADAS AO JOGO FURBOT**. 2020. 15 f. TCC (Doutorado) - Curso de Curso de Bacharel em Ciência da Computação, Universidade Regional de Blumenau (Furb), Blumenau, 2020.

COLODETTE, Fábio Louzada. **Desenvolvendo um aplicativo móvel de assistência à leitura para deficientes visuais através de visão computacional e OCR**. 2023. 62 f. Trabalho de Conclusão de Curso (Bacharelado em Sistemas de Informação) – Instituto Federal do Espírito Santo, Campus Cachoeiro de Itapemirim, 2023.

COSTA, Rafael F.; SILVA, Mariana T.; ANDRADE, Vitor L. **Desenvolvimento de aplicativo com feedback auditivo em tempo real para deficientes visuais.** Anais do Congresso Nacional de Tecnologia Assistiva e Acessibilidade, v. 4, p. 88–95, 2022.

FAIRBANKS, Alexandre de Serpa Pinto. **O direito de acesso à cultura dos cegos e das pessoas com deficiência visual.** 2023. Dissertação (Mestrado em Direito) – Universidade do Estado do Rio de Janeiro, Rio de Janeiro, 2023.

FARIA, Aline C.; SANTOS, Eduardo V.; LOPES, Gustavo M. **Aplicativo móvel para apoio à mobilidade de deficientes visuais utilizando inteligência artificial embarcada.** *Revista Brasileira de Inovação Tecnológica em Saúde*, v. 12, n. 2, p. 55–64, 2022.

FERREIRA, Bruno C.; ALMEIDA, Tiago M.; SANTOS, Ana R. **Integração de síntese de voz em aplicativos móveis para acessibilidade.** *Revista Brasileira de Informática na Educação*, v. 29, n. 1, p. 119–134, 2021.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep Learning**. Cambridge: MIT Press, 2016.

INSTITUTO BRASILEIRO DE GEOGRAFIA E ESTATÍSTICA (IBGE). **Cartilha do Censo, Pessoas com Deficiência,** 2010. Disponível em: https://inclusao.enap.gov.br/wp-content/uploads/2018/05/cartilha-censo-2010-pessoas-com-deficienciareduzido-original-eleitoral.pdf. Acesso em: 15 set. 2024.

KRIZHEVSKY, Alex; SUTSKEVER, Ilya; HINTON, Geoffrey E. **ImageNet classification with deep convolutional neural networks.** In: *Advances in Neural Information Processing Systems*, [S.l.], v. 25, 2012.

LECUN, Yann; BENGIO, Yoshua; HINTON, Geoffrey. **Deep learning.** *Nature*, v. 521, n. 7553, p. 436–444, 2015.

LIMA, Renata C.; MELO, João P.; ROCHA, Leandro T. **Uso de tecnologias móveis para acessibilidade de pessoas com deficiência visual em espaços urbanos.** *Revista Brasileira de Computação Aplicada*, v. 12, n. 1, p. 89–105, 2020.

MILANO, Danilo de; HONORATO, Luciano Barrozo. **Visão computacional.** Campinas: Faculdade de Tecnologia, Universidade Estadual de Campinas, 2010.

MONTARDO, Sandra Portella et al. **Acessibilidade digital em blogs: limites e possibilidades para socialização on-line de Pessoas com Necessidades Especiais (PNE).** *EPTIC On-line: Revista de Economía Política de las Tecnologías de la Información y Comunicación*, v. 10, n. 1, p. 1–16, 2008.

MONTENEGRO, Natalia Pereira. **Cegos e invisíveis no contexto da cultura da imagem: um estudo sobre hábitos de consumo de deficientes visuais no Rio de Janeiro.** 2015. Trabalho de Conclusão de Curso (Graduação em Comunicação – Habilitação em Publicidade e Propaganda) – Escola de Comunicação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2015.

MOURA, Felipe R.; CARDOSO, Bruno H.; GOMES, Carla M. **Tecnologias assistivas baseadas em voz para apoio à mobilidade urbana de deficientes visuais.** *Revista de Computação Aplicada*, v. 15, n. 1, p. 55–67, 2020.

NEVES, Luiz Antônio Pereira; VIEIRA NETO, Hugo; GONZAGA, Adilson. **Avanços em visão computacional.** Curitiba, PR: Omnipax, v. 1, 2012.

OLIVEIRA, G. T. L. de; MORAES, A. C. de. **A audiodescrição (AD) na formação estética de futuros(as) pedagogos(as).** *Revista Linguagem em Foco*, Fortaleza, v. 15, n. 2, p. 47–65, 2023. Disponível em: https://doi.org/10.46230/2674-8266-15-10598.

OLIVEIRA JÚNIOR, Reginaldo de Jesus; DINIZ, Ivando Severino. **Desenvolvimento de um sistema de reconhecimento de objetos com inteligência artificial para assistência a deficientes visuais utilizando redes neurais convolucionais.** In: *IX Workshop Integração de Saberes Ambientais*, Sorocaba, SP, 2024.

PASSERINO, Liliana Maria; MONTARDO, Sandra Portella. **Inclusão social via acessibilidade digital: proposta de inclusão digital para pessoas com necessidades especiais.** In: *E-Compós*, 2007.

PEREIRA, Júlia F.; RIBEIRO, Matheus A.; NASCIMENTO, Luiz G. **Implementação de um sistema de reconhecimento de objetos com inteligência artificial no Unity.** In: *Simpósio de Aplicações em Desenvolvimento de Software Educacional*, v. 7, p. 112–118, 2021.

SANDLER, Mark et al. MobileNetV2: **Inverted Residuals and Linear Bottlenecks.** In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.

SANTOS, Karine; OLIVEIRA, Ítalo R.; FONSECA, Lucas H. **Aplicativo móvel com inteligência artificial embarcada para auxiliar deficientes visuais no reconhecimento de objetos.** *Anais do Congresso de Inovação em Engenharia*, v. 5, n. 1, p. 122–129, 2022.

SANTOS, Valéria Nunes dos. **Reconhecimento de objetos em uma cena utilizando redes neurais convolucionais.** 2018. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) – Universidade Tecnológica Federal do Paraná, Medianeira, 2018.

SCHLÜNZEN, E. T. M. **A tecnologia como inclusão de Pessoas com Necessidades Especiais (PNE).** In: PELLANDA, N. et al. *Inclusão digital: tecendo redes afetivas/cognitivas*. Rio de Janeiro: DP&A, 2005. p. 195–210.

SILVA, Amanda T. da; MOURA, Felipe R.; CARDOSO, Bruno H. **Desenvolvimento de aplicativos assistivos com Unity para pessoas com deficiência.** *Revista Eletrônica de Iniciação Científica*, v. 18, n. 1, p. 76–85, 2020.

SILVA, Rodrigo Emerson Valentim da. **Um estudo comparativo entre redes neurais convolucionais para a classificação de imagens.** 2018. Trabalho de Conclusão de Curso (Bacharelado em Sistemas de Informação) – Universidade Federal do Ceará, Quixadá, 2018.

TORRES, Elisabeth Fátima; MAZZONI, Alberto Angel; MELLO, Anahi Guedes de. **Nem toda pessoa cega lê em Braille nem toda pessoa surda se comunica em língua de sinais.** *Educação e Pesquisa*, v. 33, p. 369–386, 2007.

UNITY TECHNOLOGIES. **Barracuda: Neural network inference library for Unity.** [S.l.], 2024. Disponível em: https://docs.unity3d.com/Packages/com.unity.barracuda@latest. Acesso em: 15 jun. 2025.

XU, Kelvin et al. Show, attend and tell: **Neural image caption generation with visual attention.** In: *INTERNATIONAL CONFERENCE ON MACHINE LEARNING – ICML 2015*, Lille, France. Proceedings [...]. New York: PMLR, 2015. p. 2048–2057. Disponível em: https://proceedings.mlr.press/v37/xuc15.html. Acesso em: 15 jun. 2025.